Travel Order Resover

Livrable du projet T-AIA-911

Réalisé par Justin MBODJE, Yasser MEKHFI, Mohamed FOFANA, Téophile CHARDON

Établissement de formation : EPITECH RENNES

Promotion 2026

1. Objectif du projet

Le but du projet est simple :
prendre une phrase en français qui décrit un trajet en train, et retourner un itinéraire calculé automatiquement entre deux gares.

Exemple :
« Je veux aller de Rennes à Lyon »
→ détecter que c'est un trajet
→ extraire Rennes et Lyon
→ calculer le plus court chemin entre ces deux gares
→ retourner le chemin et la distance.

Le point important n'était pas juste de faire fonctionner la solution, mais de construire un système qui permette de comparer différentes approches NLP proprement, sans tout mélanger.

2. Problème technique réel

Il y a deux parties très différentes dans ce projet :

1. Comprendre une phrase en langage naturel (incertain, ambigu).
2. Calculer un plus court chemin dans un graphe (déterministe, formel).

Le vrai défi est la transition entre les deux.

Le NLP produit du texte ou des entités.
L'algorithme de graphe attend des identifiants précis (`station_id`).

Si le mapping est faux, tout le reste est faux.

Donc le cœur du problème, ce n'est pas juste "détecter une ville", c'est la transformer correctement en nœud du graphe.

3. Architecture choisie

On a structuré le système comme un pipeline clair :

1. Classification d'intention
2. Extraction des gares
3. Validation par rapport au référentiel
4. Chargement du graphe
5. Dijkstra
6. Retour du résultat
7. (Optionnel) visualisation sur carte

Chaque étape fait une seule chose.

Ce point était volontaire.

On voulait pouvoir changer le NLP sans toucher au graphe.

Et changer le graphe sans toucher au NLP.

4. Données utilisées

4.1 Gares

Les gares sont stockées dans un fichier `stations.csv` :

- station_id
- station_name
- city
- latitude
- longitude

Le `station_id` est la clé interne.

Tout le système repose dessus.

Le NLP ne retourne pas du texte libre, il retourne un `station_id`.

C'est ce qui permet d'enchaîner avec le graphe.

4.2 Graphe

Le réseau est stocké dans `edges.csv` :

- from_station_id
- to_station_id
- distance_km

En mémoire, c'est une liste d'adjacence classique :

station -> [(voisin, distance)]

Rien de complexe ici.

Le but était d'avoir quelque chose de lisible et stable.

5. Algorithme de plus court chemin

On a implémenté Dijkstra à la main.

Pourquoi Dijkstra ?

> poids positifs uniquement

> comportement déterministe

> simple à tester

>largement suffisant pour la taille du graphe

L'objectif n'était pas d'optimiser la recherche, mais d'avoir un solveur fiable pour servir de base de comparaison.

Le path finding n’est pas la partie expérimentale du projet.

Il sert de référence stable.

6. Approches NLP implémentées

On a intégré plusieurs stratégies.

6.1 Rule-based

Approche simple :

> normalisation du texte

> recherche de noms de villes présents dans `stations.csv`

>prise des deux premières occurrences

C’est la baseline.

Elle est simple, rapide, et totalement déterministe.

6.2 spaCy NER

On utilise spaCy pour détecter des entités de type lieu.

Ensuite on fait un mapping vers `station_id`.

Important :

spaCy ne “résout” pas la gare.

Il détecte juste un texte.

La résolution vers le référentiel est faite par nous.

6.3 HuggingFace NER

Même logique que spaCy, mais avec un modèle CamemBERT.

Encore une fois :

le modèle détecte une entité,

mais le mapping vers la base de gares reste notre responsabilité.

7. Dataset d’évaluation

On a utilisé des datasets annotés :

Chaque ligne contient :

- texte
- intention attendue
- departure_gt
- arrival_gt

On a un dataset de 10 000 exemples.

Les datasets sont versionnés dans le repo, donc reproductibles.

Les évaluations sont automatisées via pytest et génèrent des rapports PDF.

Ça permet de lancer toutes les stratégies sur les mêmes données et comparer proprement.

8. Métriques utilisées

8.1 Exact-match

Un cas est correct si :

- l'intention est correcte
- la gare de départ est correcte
- la gare d'arrivée est correcte

C'est strict.

Si une seule gare est fausse → échec.

C'est volontairement exigeant.

8.2 Precision / Recall / F1

On calcule aussi :

- précision
- rappel
- F1

Ça permet de voir si une stratégie :

- détecte trop de trajets
- ou en rate trop

8.3 Métriques temporelles

On mesure :

- moyenne
- médiane
- p95

Ça permet de comparer le coût des modèles.

Attention :
les modèles NLP ont un effet “cold start” au premier appel.
Donc les premières mesures peuvent être biaisées.

9. Ce qu’on a réellement mesuré

Important :

On mesure surtout :

- la qualité d’extraction
- la capacité à produire une route valide

On ne mesure pas :

- si le chemin est réellement optimal par rapport à une vérité terrain externe
- si la distance correspond à une vraie distance SNCF

Le graphe est fonctionnel, mais pas benchmarké contre des données officielles.

10. Limites identifiées

10.1 NLP

- Rule-based fragile aux formulations nouvelles.
- NER dépend fortement du mapping vers `station_id`.
- Ambiguïtés possibles (Lille vs île, Rennes vs reine, etc.).

10.2 Données

- Dataset en partie synthétique.
- Pas d'annotation du chemin attendu.
- Exact-match très strict.

10.3 Expérimental

- Toutes les stratégies ne sont pas évaluées exactement de la même manière.
- Effets de cache possibles sur les temps.

11. Ce que le projet apporte vraiment

Le point fort n'est pas la performance brute.

Le point fort est :

- architecture propre
- séparation NLP / graphe
- automatisation des évaluations
- datasets versionnés
- comparaisons reproductibles

On a construit une base solide pour comparer des approches.

12. Conclusion

Le Travel Order Resolver est un système modulaire qui permet de tester différentes stratégies NLP dans un cadre contrôlé.

Le graphe et Dijkstra servent de socle stable.
Les briques NLP sont interchangeables et évaluées automatiquement.

Le système fonctionne de bout en bout, mais reste expérimental.
Les principales limites concernent la résolution d'entités et la définition des métriques.

Le projet constitue une base cohérente pour continuer à améliorer la compréhension des requêtes de trajet et la comparaison des modèles NLP.